ISC4 Mini Project-Instruction on how to use the program

Biyao WANG
Saad Bin Asad
Responsable, Enseignant: Bonnifait Philippe

January 2025

1 Acceleration_Open_loop

1.1 Constants and Parameter Calculations

```
vf_est = vf;
Tf = L0 / (2 * vf_est); % Time calculation
af = 2 * vf_est^2 / L0; % Acceleration calculation
```

- `vf_est`: Estimated target speed.
- `Tf`: Time required for the vehicle to accelerate from the initial position to the insertion point, based on the lane length `L0` and target speed `vf_est`.
- `af`: Constant acceleration during the acceleration process.

1.2 Open-Loop Logic: Determining Insertion Conditions

1.2.1 Checking for a Stopped Vehicle

```
if (demar == 0)
```

If the vehicle is stopped (`demar == 0`), the script selects the insertion strategy based on the number of vehicles in the queue (`N_file`).

1.2.2 Case 1: No Vehicles in the Queue

```
if N_file == 0
    t0 = t;
    demar = 1;
    N_suivi = 0;
    disp('We''re all alone!');
end
```

The vehicle starts accelerating directly, without considering insertion into a queue.

1.2.3 Case 2: One Vehicle in the Queue

```
elseif N_file == 1
```

- Computes the estimated position of the queued vehicle after `Tf` seconds (`x_vehi`).
- Determines whether to insert in front of, behind, or wait for a safer gap relative to the queued vehicle.

1.2.4 Case 3: Multiple Vehicles in the Queue

```
elseif N_file > 1
```

- Iterates through pairs of vehicles in the queue, calculating the gap between them (`Deltax`).
- Checks if the gap is sufficient (`Deltax > D0 + Dsar + Dsav`).
- If a suitable gap is found, computes the insertion time and position and adjusts the acceleration logic.

1.3 Waiting Logic

```
if (t < t0) && (~exist('flag_wait', 'var') || flag_wait == 0)
    disp('Waiting for the starting time...');
    flag_wait = 1;
end
```

If the current time `t` has not yet reached the starting time `t0`, the script outputs a waiting message.

1.4 Open-Loop Acceleration Phase

```
if t >= t0 && t < t0 + Tf
    acc = acc_B0; % Apply constant acceleration
```

- During the time interval $[t_0, t_0 + T_f]$, the vehicle accelerates at a constant rate `acc_B0`.
- Outputs the current time, position, and acceleration.

1.5 Ending the Open-Loop Phase

```
if t >= t0 + Tf
    acc = 0; % Stop accelerating
```

- When the time exceeds the open-loop phase (`t0 + Tf`), the vehicle stops accelerating.
- The flag `open_loop_done` indicates the transition to the closed-loop phase.

1.6 Summary of Script Functionality

- This script controls the vehicle's acceleration in an open-loop manner to insert into a traffic queue.
- Based on the relative position of the vehicle and the queue, it dynamically determines the insertion strategy (direct insertion, delayed waiting, or choosing the best gap).
- The acceleration phase uses constant acceleration and stops once the target position is reached.
- It supports checking insertion conditions for single or multiple vehicles in the queue and outputs detailed information about the current phase of acceleration.

2 Acceleration_Closed_loop

The closed-loop acceleration part uses the open-loop script as a prerequisite. As the necessary conditions are fulfilled, the car transitions from the open-loop to the closed loop and hence the script for the closed-loop is relatively simple

2.1 Display message:

```
disp('We are in the closed loop!!')
```

This part is used to displaying a message on the command window, indicating the car has entered a closed-loop.

2.2 Control logic:

```
if (t > t0 + Tf)
    if (abs(pos+L0/2) < vit*deltaT/2)
     disp('We have passed -L0/2. We can insert.');
    end
    T5_percent = L0 / (4 * vf_est); % T5 %
    lambda = 6 / T5_percent; % Derivative gain
    mu = (3 / T5_percent)^2; % Proportional gain
    Dxx = pos - (File(N_suivi+1).pos_est + Dsar + D0); % Safety margin
    Li = D0 + Dxx + Dsav; % Target offset for closed-loop insertion
    e = File(N_suivi).pos_est - Li - pos; % Position error
    e_dot = vf_est - vit; % Speed error
    acc = lambda * e_dot + mu * e;
    % Apply saturation
    %acc = max(decel_max, min(acc, accel_max)); % Clamp acc within saturation limits
end
```

Here, the first if-loop ensures that the closed-loop acceleration part is executed after the open-loop part has been executed. The second message is used to display when the appropriate time has been reached for insertion. Finally, the rest of code controls the logic of the closed-loop part as derived earlier in the theoretical section. The commented part is that of the Additional Work section, which can be later uncomment-ed to add to the working script.

3 Simufile

This script simulates vehicle insertion into a traffic queue under various conditions,provides a framework for testing different control strategies for vehicle insertion in a simulated environment. It includes initialization, main loop execution for the simulation, and optional visualization of results. The simulation involves:

1.Open-loop acceleration.

2.Handling noisy and accurate sensor data.

3.Switching to closed-loop control as needed.

4.Emergency stop handling to prevent overshooting the acceleration lane.

When we want to test the $Acceleration_example.m$, we set $question = 0$. When $Display_fig = 1$ then we can get a Simulation Window and the Function image of acceleration and speed versus time, which provides intuitive spatial layout information for the insertion process, showing the results of the vehicle insertion into the traffic flow and provides dynamic information in the time dimension for analyzing the performance of vehicle motion control, including acceleration, speed, and target matching.

And the same when we set $question = 1$ to use the open loop strategy without noise, set $question = 2$ to use the open loop strategy with noise, set $question = 3$ to use the closed loop strategy. For the additional work, we simply remove the syntax for the following comment from the closed-loop script and make it a part of the actual script as follows:

```
% Apply saturation
    acc = max(decel_max, min(acc, accel_max)); % Clamp acc within saturation limits
```

Figure 1: Simulation Window

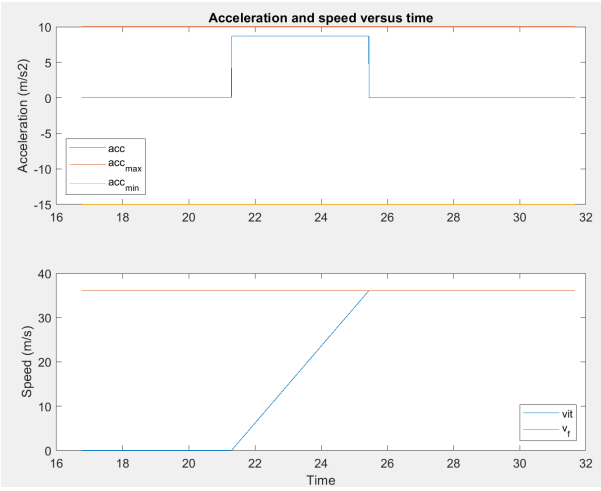

Figure 2: Function image of acceleration and speed versus time

4 Launch_Simufile

This script automates running simulations, logs the outputs, analyzes the results, and visualizes the findings through bar and pie charts. Here's a detailed breakdown of its functionality:

We record all console output to the file *diary*.

Here we add some codes in order to :

1. Reads the log file (diary) and splits the output into individual simulation logs using the separator.

2. Analyzes the logs to classify each simulation result into one of three categories,Prints the counts for each category to the console.

3. Prints the counts for each category to the console.

4. Visualization: Creates a bar chart and a pie chart showing the number of simulations in each category.

```
% Read the diary file
log_data = fileread('diary'); % Ensure the diary file is in the same folder

% Split the log into individual simulations
simulations = regexp(log_data, '--------------------------------', 'split');

% Initialize counters for each category
unsuccessful_count = 0;
unsafe_success_count = 0;
safe_success_count = 0;

% Loop through each simulation log
for i = 1:length(simulations)
    sim = simulations{i};

    % Check if it's an unsuccessful insertion
    if contains(sim, 'Emergency stop')
        unsuccessful_count = unsuccessful_count + 1;
    % Check if it's a successful insertion but safety distances not respected
    elseif contains(sim, 'Successful insertion!') && contains(sim, 'Safety
        distances are not respected!')
```

```
        unsafe_success_count = unsafe_success_count + 1;
    % Check if it's a successful insertion with safety distances respected
    elseif contains(sim, 'Successful insertion!')
        safe_success_count = safe_success_count + 1;
    end
end

% Display the counts
disp(['Unsuccessful insertions: ', num2str(unsuccessful_count)]);
disp(['Successful but unsafe insertions: ', num2str(unsafe_success_count)]);
disp(['Successful and safe insertions: ', num2str(safe_success_count)]);

% Prepare data for visualization
categories = {'Unsuccessful', 'Unsafe Success', 'Safe Success'};
counts = [unsuccessful_count, unsafe_success_count, safe_success_count];

% Create a bar chart
figure;
bar(counts);
set(gca, 'xticklabel', categories);
xlabel('Category');
ylabel('Number of Simulations');
title('Simulation Results');
grid on;

% Create a pie chart
figure;
pie(counts, categories);
title('Simulation Results Distribution');
```

Note: The diary must be deleted/reset each time the script is run, otherwise the results from the previous simulations will also be accumulated into the new simulation.